ASSIGNMENT 13.5

Name:Thanay N

Hallticket.no:2303A51424

batch.no:21

Task Description #1 (Refactoring – Removing Global Variables)

• Task: Use AI to eliminate unnecessary global variables from the]code.

• Instructions:

o Identify global variables used across functions.

o Refactor the code to pass values using function parameters.

o Improve modularity and testability.

• Sample Legacy Code:

```
rate = 0.1
def calculate_interest(amount):   return amount * rate
print(calculate_interest(1000))
```

code screenshot:

```python
lab27-2-26.py > ...
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
#identify the global variable in the code above
'''The global variable in the code above is `rate`.
It is defined outside of the function `calculate_interest`
and can be accessed and used within the function.'''
#eliminate the unnessary global variable and refractor the code to pass values using function parameters
def calculate_interest(amount, rate):
    return amount * rate
#give example outputs
print(calculate_interest(1000, 0.1))  # Output: 100.0
print(calculate_interest(500, 0.2))   # Output: 100.0
```

Output:

```
100.0
100.0
All test cases passed!
```

Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
- Focus Areas: Readability

  Logical simplification

  Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

print("Very Good")

else:

if score >= 60:

print("Good")

else:

print("Needs Improvement")

code screenshot:

```
lab27-2-26.py > ...
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
Expected Outcome:
#refactor deeply nested if-elif-else logic into a cleaner structure improve readability and maintainability.
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:
    print("Very Good")
elif score >= 60:
    print("Good")
else:    print("Needs Improvement")
#flattened logic using guard clauses or a mapping-based approach
score = 78
if score >= 90:
    print("Excellent")
    exit()
if score >= 75:
    print("Very Good")
    exit()
if score >= 60:
    print("Good")
    exit()
print("Needs Improvement")
#Using a mapping-based approach
score = 78
def get_grade(score):
    if score >= 90:
        return "Excellent"
    elif score >= 75:
        return "Very Good"
    elif score >= 60:
        return "Good"
    else:
        return "Needs Improvement"
grade = get_grade(score)
print(grade)
```

Output:

```
Very Good
Very Good
Excellent
Very Good
Good
Needs Improvement
```

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.
- Focus Areas:

  DRY principle

  Context managers

  Function reuse

Legacy Code:

```
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
```

f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

lab27-2-26.py > ...

```python
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
'''refactor repeated file open/read/close
give me resuable function with open() and parameters
logic DRY principle'''
def read_file(filename):
    with open(filename) as f:
        return f.read()
print(read_file("data1.txt"))
print(read_file("data2.txt"))
```

Task 4 (Optimizing Search Logic)

• Task: Refactor inefficient linear searches using appropriate

data structures.

• Focus Areas:

o Time complexity

o Data structure choice

Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:

if u == name:

found = True

print("Access Granted" if found else "Access Denied")

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Code screenshot:

```python
lab27-2-26.py > ...
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
'''refactor this code with efficient suitable search data structures and algorithms
minimum complexity should be O(1)'''
# Using a set for O(1) average lookup time
users_set = set(users)
if name in users_set:
    print("Access Granted")
else:
    print("Access Denied")

'''Use of sets or dictionaries with complexity justification'''
# Sets provide O(1) average time complexity for membership testing, which is more efficient than the O(n)
# linear search in a list.
# This is especially beneficial when dealing with large datasets where performance matters.
```

Output:

```
Enter username: saisharan
Access Denied
Enter username: admin
Access Granted
PS C:\Users\phani\Downloads\saisharan>
```

Task 5 (Refactoring Procedural Code into OOP Design)

• Task: Use AI to refactor procedural code into a class-based design.

• Focus Areas:

o Object-Oriented principles

o Encapsulation

Legacy Code:

salary = 50000

tax = salary * 0.2

net = salary - tax

print(net)

Expected Outcome:

o A class like EmployeeSalaryCalculator with methods and attributes.

Code screenshot:

```
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
'''refactor procedural code into a class-based design using
object oriented programming principles like encapsulations.
Create a class called Employee with attributes for salary and tax rate, and a method
(EmployeeSalaryCalculator)to calculate net salary.
 '''
class Employee:
    def __init__(self, salary, tax_rate):
        self.salary = salary
        self.tax_rate = tax_rate

    def EmployeeSalaryCalculator(self):
        tax = self.salary * self.tax_rate
        net_salary = self.salary - tax
        return net_salary
employee = Employee(50000, 0.2)
net_salary = employee.EmployeeSalaryCalculator()
print(net_salary)
```

Output:

```
40000.0
All test cases passed!
```

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling

large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Code screenshot:

```python
# Refactored Code using Mathematical Formula
n = 999998  # Largest even number less than 1,000,000
total = n * (n + 2) // 4  # Sum of first n even numbers
print(total)
# Refactored Code using List Comprehension
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
print(total)
# Time Comparison
import time
# Timing the original code
start_time = time.time()
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
end_time = time.time()
print(f"Original code time: {end_time - start_time:.6f} seconds")
# Timing the mathematical formula code
start_time = time.time()
n = 999998
total = n * (n + 2) // 4
end_time = time.time()
print(f"Mathematical formula code time: {end_time - start_time:.6f} seconds")
# Timing the list comprehension code
start_time = time.time()
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
end_time = time.time()
print(f"List comprehension code time: {end_time - start_time:.6f} seconds")
```

Output:

```
Mathematical formula code time: 0.000000 seconds
List comprehension code time: 0.090078 seconds
```

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas:

o Functional-style refactoring

o Predictability

Legacy Code:

data = []

def add_item(x):

data.append(x)

add_item(10)

add_item(20)

print(data)

Expected Outcome:

o Refactored function returning values instead of mutating globals.

Code screenshot:

```
def add_item(data, x):
    return data + [x]
data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

Output:

```
PS C:\Users\
[10, 20]
```

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation

rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

print("Must contain uppercase")

else:

print("Must contain digit")

else:

print("Password too short")

Expected Outcome:

o Separate validation functions with clear responsibility

code screenshot:

```python
def is_valid_length(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)

password = input("Enter password: ")
if is_valid_length(password):
    if has_digit(password):
        if has_uppercase(password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Output:

```
Enter password: 1434
Password too short
```

Task 8 (Refactoring Complex Input Validation Logic)

• Task: Use AI to simplify and modularize complex validation rules.

• Focus Areas:

o Readability

o Testability

Legacy Code:

password = input("Enter password: ")

if len(password) >= 8:

if any(c.isdigit() for c in password):

if any(c.isupper() for c in password):

print("Valid Password")

else:

```
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
```

Expected Outcome:

o Separate validation functions with clear responsibility

code screenshot :

sers > phani > OneDrive > Desktop > Btech > 3-2 > assign1.1.py > ...

```python
def is_valid_length(password):
    return len(password) >= 8

def has_digit(password):
    return any(c.isdigit() for c in password)

def has_uppercase(password):
    return any(c.isupper() for c in password)
def validate_password(password):
    if not is_valid_length(password):
        return "Password too short"
    if not has_digit(password):
        return "Must contain digit"
    if not has_uppercase(password):
        return "Must contain uppercase"
    return "Valid Password"
password = input("Enter password: ")
validation_result = validate_password(password)
print(validation_result)

# Test cases for the password validation functions
def test_is_valid_length():
    assert is_valid_length("password") == True
    assert is_valid_length("pass") == False

def test_has_digit():
    assert has_digit("password1") == True
    assert has_digit("password") == False

def test_has_uppercase():
    assert has_uppercase("Password") == True
    assert has_uppercase("password") == False

# Run the tests
test_is_valid_length()
test_has_digit()
test_has_uppercase()
print("All tests passed!")
```

Output screenshot:

```
Enter password: saisharan
Must contain digit
All tests passed!
PS D:\Devops>
```